



REPUBLIQUE DU BENIN



MINISTRE DE L'ENSEIGNEMENT
SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE

UNIVERSITE D'ABOMEY CALAVI (UAC)

INSTITUT DE FORMATION ET DE RECHERCHE EN
INFORMATIQUE

*****.

THÈME

De l'automate fini à la machine universelle

Membre du groupe n° 6
ADJOVI Ruben
LOKOSSOU Ayédon Iréné Sang Froid

Sous la supervision : Dr. Mêton ATINDEHOU

Jour 1 Étage régulier : automates finis et transducteurs

$\Sigma = \{a, o, r\}$ $L_1 = \{ w \mid w \text{ contient le facteur } or \}$

Q1.1 Expression régulière

Donnons une regex dénotant exactement L_1 :

$$L_1 = (a + o + r)^* \cdot or \cdot (a + o + r)^*$$

Justification : L'expression $(a+o+r)^*$ génère toute suite finie (y compris ε) sur l'alphabet $\Sigma = \{a, o, r\}$. Le facteur central obligatoire or impose que les lettres o et r apparaissent consécutivement. En les encadrant des deux côtés par $(a+o+r)^*$, on capture exactement tous les mots contenant ce facteur et aucun autre.

Q1.2 Du non-déterminisme à l'AFD minimal

a) Construction de l'AFN

L'AFN naturel pour « contient or » possède trois états :

q_0 : état initial — non encore engagé sur le facteur

q_1 : vient de lire un o susceptible de commencer le facteur

q_2 : a lu or complet — état final, $F = \{q_2\}$

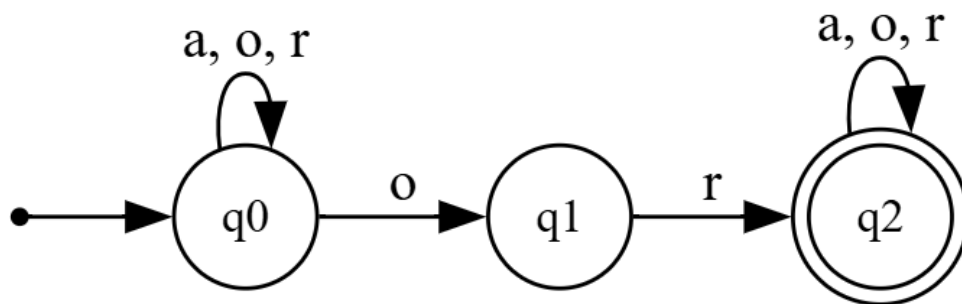


Figure 1 Représentation graphique de l'AFN

Table de transition de l'AFN :

État	a	o	r
q_0	q_0	q_0, q_1	q_0
q_1	—	—	q_2
q_2 (final)	q_2	q_2	q_2

Non-déterminisme visible en q_0 sur o : on peut rester en q_0 ou aller en q_1 .

b) Déterminisation (construction des sous-ensembles)

On part de $S_0 = \{q_0\}$ et on ferme par transition :

État	Sur a	Sur o	Sur r
$S_0 = \{q_0\}$	S_0	S_1	S_0
$S_1 = \{q_0, q_1\}$	S_0	S_1	S_2
$S_2 = \{q_0, q_2\}$ (Final car $q_2 \in S_2$)	S_2	S_3	S_2
$S_3 = \{q_0, q_1, q_2\}$ (Final car $q_2 \in S_3$)	S_2	S_3	S_2

États finaux du AFD = états contenant q_2 , soit S_2 et S_3 .

c) Minimisation (raffinement de Moore)

Pour cela recherchons le même ensemble de mots avec le moins d'états possible dans l'automate.

Partitionnons en deux classes : I pour les états non-finaux et II pour les états finaux :

$$I = \{S_0, S_1\} \text{ (non-finaux)}$$

$$II = \{S_2, S_3\} \text{ (finaux)}$$

Calculons les images des éléments de I et partitionnons en fonction :

État (I)	a	o	r
S_0	S_0	S_1	S_0
S_1	S_0	S_1	S_2

En remplaçant les états par leurs classes :

État	a	o	r
S_0	I	I	I
S_1	I	I	II

Il y a donc 2 formes d'images : (I, I, I) ou (I, I, II).

$$\rightarrow I = I_1 \cup I_2 \text{ avec } I_1 = \{S_0\} \text{ et } I_2 = \{S_1\}$$

On a maintenant 3 ensembles : I_1 , I_2 et II.

Raffinons II :

État (II)	a	o	r
S_2	S_2	S_3	S_2
S_3	S_2	S_3	S_2

État	a	o	r
S ₂	II	II	II
S ₃	II	II	II

S₂ et S₃ ont les mêmes images $\rightarrow$ II = {S₂, S₃} reste groupé.

Résultat : 3 classes = 3 états. L'automate est minimisé. On pose A = {S₀}, B = {S₁}, C = {S₂, S₃}.

d) AFD minimal

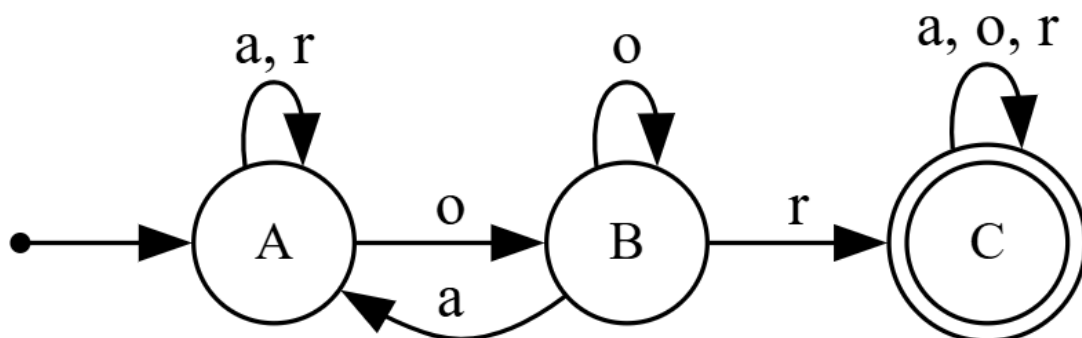


Figure 2 AFD minimal

État	a	o	r
A (initial)	A	B	A
B	A	B	C
C (Final)	C	C	C

Q1.3 Facteur vs sous-séquence

Soit $L'_1 = \{ w \mid w \text{ contient un } o \text{ suivi, pas forcément immédiatement, d'un } r \}$

Regex :

$$L'_1 = (a+o+r)^* \cdot o \cdot (a+o+r)^* \cdot r \cdot (a+o+r)^*$$

Justification et différence avec L_1 : Dans L_1 , les lettres o et r forment un facteur, signifiant qu'elles doivent être collées l'une à l'autre sans interruption soit (...or...). Dans L'_1 , elles forment une **sous-séquence**, ce qui signifie que le o doit simplement apparaître avant le r dans le mot peu importe le nombre de lettres intermédiaires soit (...o...r...).

Preuve de l'inclusion $L_1 \subseteq L'_1$: Soit un mot $w \in L_1$. Par définition de l'expression régulière de L_1 , il existe deux chaînes $u, v \in \Sigma^*$ telles que $w = u \cdot "or" \cdot v$. Cette écriture équivaut strictement à $w = u \cdot "o" \cdot \varepsilon \cdot "r" \cdot v$. Comme le mot vide ε appartient par définition à Σ^* , la structure de w répond parfaitement au motif de l'expression régulière de L'_1 (un o, une chaîne intermédiaire vide, un r). Par conséquent, tout mot contenant le facteur or contient de facto la sous-séquence o...r, prouvant ainsi que $L_1 \subseteq L'_1$.

Mot de $L'_1 \setminus L_1$: arooaroar car pour ce mot, le caractère o est bien positionné avant le r, mais la présence du a empêche la formation du facteur « or » dans la partie « oar ».

E1.1 Détecteur AFD

La table de transitions définit les 3 états de l'automate :

- En A on n'a rien lu d'utile. Sur o on passe en B, sinon on reste en A.

- En B on vient de lire un o et on est « prêt ». Sur r on bascule en C (trouvé !), sur o on reste en B, sur a on retombe en A.
- En C le facteur or est détecté. On reste en C quoi qu'il arrive, impossible de "désaccepter".

run parcourt le mot lettre par lettre en suivant la table. Si à un moment une lettre n'a pas de transition prévue, elle renvoie None (mot rejeté proprement, sans crash).

accepts appelle simplement run et vérifie si l'état final atteint est dans l'ensemble des états acceptants {C}.

```
_DFA_OR = DFA(
    transitions={
        ("A", "a"): "A",
        ("A", "o"): "B",
        ("A", "r"): "A",
        ("B", "a"): "A",
        ("B", "o"): "B",
        ("B", "r"): "C",
        ("C", "a"): "C",
        ("C", "o"): "C",
        ("C", "r"): "C",
    },
    start="A", accept={"C"}, alphabet={"a", "o", "r"},
)
```

detector.py : table de l'AFD "or"

```
def run(self, w: str):
    state = self.start
    for c in w:
        state = self.transitions.get((state, c))
        if state is None:
            return None
    return state

def accepts(self, w: str) -> bool:
    return self.run(w) in self.accept
```

E1.2 Minimisation

minimize vérifie si on peut fusionner des états qui se comportent exactement pareil.

On commence par séparer les états en deux groupes : ceux qui acceptent et ceux qui n'acceptent pas. Ensuite on regarde, pour chaque groupe, si tous ses membres réagissent de la même façon sur chaque lettre. Si deux états d'un même groupe mènent à des groupes différents, on les sépare. On répète jusqu'à ce que plus rien ne bouge.

Pour l'AFD de or, on retrouve bien **3 groupes = 3 états** confirmant les résultats précédents.


```

def minimize(self) -> "DFA":
    states, trans = self._completed()

    # Partition initiale : finaux vs non-finaux
    finals = frozenset(s for s in states if s in self.accept)
    non_finals = frozenset(states - finals)
    partition = set()
    if finals:
        partition.add(finals)
    if non_finals:
        partition.add(non_finals)

    changed = True
    while changed:
        changed = False
        new_partition = set()
        for block in partition:
            groups = {}
            for s in block:
                sig = tuple(
                    next((i for i, b in enumerate(partition) if trans.get((s, a)) in b), -1)
                    for a in sorted(self.alphabet)
                )
                groups.setdefault(sig, set()).add(s)
            for grp in groups.values():
                new_partition.add(frozenset(grp))
            if len(groups) > 1:
                changed = True
        partition = new_partition

    rep = {s: min(block) for block in partition for s in block}

```

```

new_trans = {}
for (s, a), t in trans.items():
    rs, rt = rep[s], rep[t]
    if rs != "__sink__" and rt != "__sink__":
        new_trans[(rs, a)] = rt
    elif rs != "__sink__":
        new_trans[(rs, a)] = rt

new_trans = {(rep[s], a): rep[t]
              for (s, a), t in trans.items()
              if rep[s] != "__sink__"}

new_accept = {rep[s] for s in self.accept if s in rep}
return DFA(new_trans, rep[self.start], new_accept, self.alphabet)

```

dfa.py : minimize (raffinement de Moore)

E1.3 Déterminisation

to_dfa transforme un AFN (qui peut avoir plusieurs chemins possibles pour une même lettre) en AFD (un seul chemin).

Le principe : chaque état du nouvel AFD représente un ensemble d'états de l'AFN. On part de l'état initial, et pour chaque lettre possible on calcule tous les états NFA qu'on pourrait atteindre. On continue jusqu'à ce qu'il n'y ait plus de nouveaux ensembles à explorer. Un état du DFA est acceptant s'il contient au moins un état acceptant de l'AFN.

```

def to_dfa(self) -> DFA:
    start_set = self._eps_closure(frozenset({self.start}))

    def name(fs): return str(sorted(fs))

    queue = [start_set]
    visited = {start_set}
    trans = {}
    accept = set()

    while queue:
        current = queue.pop(0)
        if any(s in self.accept for s in current):
            accept.add(name(current))
        for a in self.alphabet:
            nxt = self._eps_closure(self._move(current, a))
            trans[(name(current), a)] = name(nxt)
            if nxt not in visited:
                visited.add(nxt)
                queue.append(nxt)

    return DFA(trans, name(start_set), accept, self.alphabet)

```

nfa.py : to_dfa (construction des sous-ensembles)

E1.4 Transducteur leet

leet_fst construit un transducteur avec un seul état qui remplace les chiffres leet par leurs lettres (4→a, 0→o, 3→e, 1→i, 5→s). Toute lettre non listée passe sans modification grâce à identity_on_missing=True.

transduce lit le mot caractère par caractère. Pour chaque caractère, si une substitution existe elle l'applique, sinon elle recopie le caractère tel quel. Le résultat est idempotent : passer le mot deux fois dans le transducteur donne le même résultat que le passer une seule fois, puisqu'après le premier passage il n'y a plus de chiffres leet à remplacer.

```

def leet_fst() -> "SequentialFST":
    transitions = {
        ("q0", "4"): ("q0", "a"),
        ("q0", "3"): ("q0", "e"),
        ("q0", "0"): ("q0", "o"),
        ("q0", "1"): ("q0", "i"),
        ("q0", "5"): ("q0", "s"),
    }

```

```
def transduce(self, w: str) -> str:
    state = self.start
    out = []
    for c in w:
        key = (state, c)
        if key in self.transitions:
            state, sym = self.transitions[key]
            out.append(sym)
        elif self.identity_on_missing:
            out.append(c)
        else:
            return None
    return "".join(out)
```

fst.py : leet_fst + transduce

Résultats des tests

tests/test_regular.py::test_detector_or PASSED	[14%]
tests/test_regular.py::test_minimisation_3_etats PASSED	[28%]
tests/test_regular.py::test_nfa_to_dfa_equivalence PASSED	[42%]
tests/test_regular.py::test_leet_idempotent PASSED	[57%]
tests/test_regular.py::test_fst_composition PASSED	[71%]
tests/test_regular.py::test_miroir_twoway PASSED	[85%]
tests/test_regular.py::test_delimiters_pda FAILED	[100%]

Bonus Jour 1 : construction de Thompson (regex → AFN)

On construit l'automate morceau par morceau, en suivant la structure de la regex. Un caractère seul, c'est juste deux états reliés par une flèche. Pour une concaténation, on colle la fin du premier automate au début du second avec une flèche silencieuse (epsilon). Pour une alternative (a|b), on crée un nouvel état de départ qui se ramifie en epsilon vers les deux automates possibles. Pour une étoile, on ajoute des flèches epsilon qui permettent de boucler zéro, une ou plusieurs fois sur le même automate. À la fin on obtient un AFN plein de transitions epsilon, qu'on peut ensuite déterminer comme d'habitude.

```

def thompson(regex: str) -> "NFA":
    """Construction de Thompson : regex -> AFN avec transitions epsilon."""
    counter = [0]

    def new_state():
        s = f"s{counter[0]}"
        counter[0] += 1
        return s

    def from_char(c):
        # Un caractère : un état initial, une transition, un état final
        s0, s1 = new_state(), new_state()
        return NFA(
            transitions={(s0, c): {s1}},
            start=s0, accept={s1}, alphabet={c}
        )

    def epsilon():
        # Mot vide : transition epsilon directe
        s0, s1 = new_state(), new_state()
        return NFA(
            transitions={(s0, ""): {s1}},
            start=s0, accept={s1}, alphabet=set()
        )

```

nfa.py : thompson (construction)

```

def test_thompson():
    from formlang.nfa import thompson
    # regex : (a|b)*b - tous les mots sur {a,b} finissant par b
    nfa = thompson("(a|b)*b")
    assert nfa.accepts("b") is True
    assert nfa.accepts("ab") is True
    assert nfa.accepts("aab") is True
    assert nfa.accepts("a") is False
    assert nfa.accepts("") is False
    assert nfa.accepts("ba") is False
    # regex : a* - zéro ou plusieurs a
    nfa2 = thompson("a*")
    assert nfa2.accepts("") is True
    assert nfa2.accepts("a") is True
    assert nfa2.accepts("aaa") is True
    assert nfa2.accepts("b") is False

```

test_regular : test thompson

```
PS C:\Users\RUBENS\Documents\Master\SEMESTRE_TWO\THEORIE DES LANGAGES\formlang_projet> python -m pytest tests/test_regular.py::test_thompson -v
===== test session starts =====
platform win32 -- Python 3.14.3, pytest-9.1.1, pluggy-1.6.0 -- C:\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\RUBENS\Documents\Master\SEMESTRE_TWO\THEORIE DES LANGAGES\formlang_projet
configfile: pyproject.toml
plugins: anyio-4.13.0
collected 1 item

tests/test_regular.py::test_thompson PASSED [100%]
===== 1 passed in 0.11s =====
```

Jour 2 Étage hors-contexte : grammaires & automate à pile

Alphabet : $\Sigma_3 = \{ a, o, r, [,], (,) \}$

Q2.1 Non-régularité par pompage

Démonstration par l'absurde que $D = \{ [^n]^n \mid n \geq 0 \}$ n'est pas régulier.

Énoncé du lemme de l'étoile

Si L est régulier, il existe une constante de pompage $p \geq 1$ telle que tout mot $w \in L$ avec $|w| \geq p$ peut s'écrire $w = xyz$ avec :

- (1) $|xy| \leq p$
- (2) $|y| \geq 1$ ($y \neq \varepsilon$)
- (3) Pour tout $i \geq 0$: $xy^iz \in L$

Preuve

Supposons que D soit un langage régulier. D'après le lemme de l'étoile, il existe une constante de pompage $p \geq 1$ associée à ce langage.

Choix du mot : Considérons le mot $w = [^p]^p$. Ce mot appartient manifestement à D et sa longueur totale $|w| = 2p$ est bien supérieure ou égale à p .

Analyse des découpes $w = xyz$ admissibles : Le lemme de l'étoile impose les deux contraintes fondamentales suivantes :

- $|xy| \leq p$

- $|y| > 0$ (soit $y \neq \varepsilon$)

Puisque la sous-chaîne xy est confinée dans les p premiers caractères de w , et que ces p premiers caractères ne contiennent exclusivement que des crochets ouvrants $[$, le facteur y est obligatoirement composé uniquement de crochets ouvrants. On peut donc poser :

$$y = [^k \text{ avec } 1 \leq k \leq p$$

Application du pompage (pour $i = 2$) :

Évaluons le mot pompé deux fois $w_2 = xy^2z$.

Le mot initial w possède exactement p crochets $[$ et p crochets $]$. Injecter un facteur y supplémentaire revient à rajouter k crochets ouvrants sans modifier les fermants. Le mot résultant est :

$$w_2 = [^{p+k}]^p$$

Ainsi comme $|y| \geq 1$, nous avons $k \geq 1$, d'où $p + k > p$. Le nombre de crochets ouvrants n'est plus égal au nombre de crochets fermants. Par conséquent, $w_2 \notin D$, ce qui contredit directement le lemme de l'étoile.

Par conséquent, le langage D n'est donc pas régulier.

Conséquence pour le projet : Le composant SingularityDetector (basé sur un AFD) possède une structure mémoire finie. Il est donc théoriquement incapable de compter les crochets pour valider l'imbrication récursive à n'importe quelle profondeur.

Q2.2 Grammaire hors-contexte

On donne une grammaire G engendrant les prompts bien parenthésés sur Σ_3 :

Règles de production de la grammaire G :

L'axiome S engendre des suites de blocs bien parenthésés à l'intérieur desquels les jetons abstraits $\{a, o, r\}$ peuvent s'insérer librement.

$$S \rightarrow S S \mid [S] \mid (S) \mid a \mid o \mid r \mid \varepsilon$$

Exemple de dérivation pour le prompt $[a(o)]$:

$$S \rightarrow [S] \rightarrow [SS] \rightarrow [aS] \rightarrow [a(S)] \rightarrow [a(o)]$$

Q2.3 Ambiguïté et désambiguïsation

Diagnostic d'ambiguïté : La grammaire G proposée est **ambiguë** : La règle de concaténation pure $S \rightarrow SS$ n'impose aucune associativité (ni à gauche, ni à droite), ce qui permet de générer plusieurs arbres de dérivation distincts pour des séquences de terminaux adjacents.

Deux arbres de dérivation distincts pour le mot aaa :

Arbre de dérivation n°1	Arbre de dérivation n°2
S	S
/ \	/ \
S S	S S
/ \	/ \
S S a	a S S
a a	a a

Grammaire non ambiguë équivalente : Pour éliminer l'ambiguïté structurelle de la concaténation, on sépare la génération des listes de blocs de celle des blocs eux-mêmes en introduisant une variable intermédiaire T (associativité à droite) :

$$S \rightarrow TS \mid T$$

$$T \rightarrow [S] \mid (S) \mid a \mid o \mid r \mid \varepsilon$$

E2.1 Automate à pile

accepts vérifie si les délimiteurs d'un mot sont bien imbriqués en utilisant une pile.

On lit le mot lettre par lettre. Chaque fois qu'on voit un ouvrant [ou (on le pose sur la pile. Chaque fois qu'on voit un fermant] ou) on regarde ce qui est au sommet : si ça correspond à l'ouvrant attendu on le retire, sinon on rejette immédiatement. Les lettres a, o, r, e sont ignorées. À la fin, si la pile est vide tout est bien fermé : on accepte. S'il reste quelque chose dedans, un ouvrant n'a pas été fermé : on rejette.

```
def accepts(self, w: str) -> bool:
    pile = []
    for c in w:
        if c in self.open:
            pile.append(c)
        elif c in self.match:
            if not pile or pile[-1] != self.match[c]:
                return False
            pile.pop()
        elif c in self.ignore:
            continue
        else:
            return False
    return len(pile) == 0
```

pda.py : accepts

E2.2 Génération bornée

generate produit tous les mots valides de longueur $\leq \text{max_len}$ à partir des règles de la grammaire.

On part du symbole de départ S et on le remplace progressivement par ses règles, un symbole à la fois. Pour éviter que $S \rightarrow SS \rightarrow SSS \rightarrow \dots$ parte à l'infini sans jamais produire de lettres, on fixe une limite sur le nombre de symboles non-terminaux autorisés dans une forme intermédiaire. Dès qu'une forme ne contient plus que des lettres et que sa longueur est dans la limite, on l'ajoute au résultat.

Code Source :

```
def generate(self, max_len: int) -> set:
    max_nt = 2 * max_len + 2
    agenda = {("S",)}
    terminaux = set()
    vus = {("S",)}
    while agenda:
        nouveau = set()
        for forme in agenda:
            nb_nt = sum(1 for s in forme if s in self.nonterminals)
            for i, sym in enumerate(forme):
                if sym in self.nonterminals:
                    for regle in self.rules[sym]:
                        nouvelle_forme = forme[:i] + regle + forme[i+1:]

                        nb_t = sum(1 for s in nouvelle_forme
                                if s not in self.nonterminals)
                        nb_nt_new = sum(1 for s in nouvelle_forme
                                      if s in self.nonterminals)

                        if nb_t > max_len or nb_nt_new > max_nt:
                            continue

                        if nouvelle_forme not in vus:
                            vus.add(nouvelle_forme)
                            if nb_nt_new == 0:
                                mot = "".join(nouvelle_forme)
                                if len(mot) <= max_len:
                                    terminaux.add(mot)
                            else:
                                nouveau.add(nouvelle_forme)
                        break
            agenda = nouveau
    return terminaux
```

cfg.py : generate

Résultats des tests

```
tests/test_regular.py::test_detector_or PASSED [ 14%]
tests/test_regular.py::test_minimisation_3_etats PASSED [ 28%]
tests/test_regular.py::test_nfa_to_dfa_equivalence PASSED [ 42%]
tests/test_regular.py::test_leet_idempotent PASSED [ 57%]
tests/test_regular.py::test_fst_composition PASSED [ 71%]
tests/test_regular.py::test_miroir_twoway PASSED [ 85%]
tests/test_regular.py::test_delimiters_pda PASSED [100%]

===== 7 passed in 0.12s =====
```

```
tests/test_cfg_nerode.py::test_cfg_engendre_des_mots_equilibres PASSED [ 50%]
tests/test_cfg_nerode.py::test_nerode_trois_classes FAILED [100%]
```

Bonus Jour 2 reconnaissance par CYK

On veut savoir si un mot appartient au langage d'une grammaire mise sous forme normale de Chomsky (chaque règle produit soit un caractère, soit exactement deux non-terminaux). On construit un tableau triangulaire : la case $[i][i]$ contient les non-terminaux qui produisent directement la lettre en position i . Puis, pour chaque sous-mot de plus en plus long, on essaie toutes les façons de le couper en deux morceaux : si une règle $A \rightarrow B C$ existe et que B engendre le morceau de gauche pendant que C engendre le morceau de droite, alors A engendre le sous-mot entier. On remonte ainsi jusqu'au mot complet ; il est accepté si le symbole de départ apparaît dans la case du haut.

```
def to_cnf(cfg: "CFG") -> "CFG":
    """Convertit une CFG en Forme Normale de Chomsky (CNF)."""
    rules = {}
    nonterminals = set(cfg.nonterminals)
    counter = [0]

    def new_nt():
        s = f"X{counter[0]}"
        counter[0] += 1
        nonterminals.add(s)
        return s
```

formlang/cfg.py :fonction to_cnf

```

def cyk(word: str, cfg: "CFG") -> bool:
    """Algorithme CYK : décide si word ∈ L(cfg). La grammaire doit être en CNF."""
    n = len(word)
    if n == 0:
        # Cas mot vide : vérifier si S → ε existe
        return any(prod == () for prod in cfg.rules.get(cfg.start, []))

    # table[i][j] = ensemble des non-terminaux qui génèrent word[i:j+1]
    table = [[set() for _ in range(n)] for _ in range(n)]

    # Remplir la diagonale : règles unitaires (A → a)
    for i, c in enumerate(word):
        for nt, prods in cfg.rules.items():
            if (c,) in prods:
                table[i][i].add(nt)

    # Remplir le reste de la table
    for length in range(2, n + 1):          # longueur du sous-mot
        for i in range(n - length + 1):      # début
            j = i + length - 1              # fin
            for k in range(i, j):            # point de coupure
                for nt, prods in cfg.rules.items():
                    for prod in prods:
                        if len(prod) == 2:
                            B, C = prod
                            if B in table[i][k] and C in table[k+1][j]:
                                table[i][j].add(nt)

```

formlang/cfg.py : fonction cyk

```

def test_cyk():
    from formlang.cfg import cyk, CFG

    # Grammaire simple en CNF : S → A B | A S, A → a, B → b
    # Langage : a^n b avec n ≥ 1
    cfg = CFG(
        rules={
            "S": [("A", "B"), ("A", "S")],
            "A": [("a",)],
            "B": [("b",)],
        },
        start="S",
        nonterminals={"S", "A", "B"},
    )
    assert cyk("ab", cfg) is True
    assert cyk("aab", cfg) is True
    assert cyk("aaab", cfg) is True
    assert cyk("b", cfg) is False
    assert cyk("a", cfg) is False
    assert cyk("ba", cfg) is False
    assert cyk("", cfg) is False

```

tests/test_cfg_nerode.py : test test_cyk

```
tests/test_cfg_nerode.py::test_cyk PASSED
```

```
[100%]
```

```
===== 1 passed in 0.10s =====
```

Jour 3 Automates d'arbres (pivot) & hash-consing

Q3.1 Déterminisme & Coût

Rappel : un BUTA est **déterministe** si, pour tout symbole f et toute séquence $(q_1, \dots, q_n)$ d'états enfants, il existe **au plus une** règle $f(q_1, \dots, q_n) \rightarrow q$ dans Δ .

Montrons que morpho_automaton et shield_automaton sont déterministes :

morpho_automaton : chaque nœud de structure (prefixes, suffixes, rest, word) cible une configuration unique d'états typés (PRE, ROOT, SUF, NIL) sans aucun chevauchement ni ambiguïté de réduction.

Symbole	États enfants requis	État produit
prefix(s)	(étiquette s)	PRE
root(s)	(étiquette s)	ROOT
suffix(s)	(étiquette s)	SUF
nil	()	NIL
prefixes(h, t)	(PRE, PREFS NIL)	PREFS
suffixes(h, t)	(SUF, SUFS NIL)	SUFS
rest(r, s)	(ROOT, SUFS NIL)	REST
word(p, r)	(PREFS NIL, REST)	WORD

shield_automaton : la fonction d'évaluation des séquences associe à chaque paire d'états enfants une unique transition déterministe (soit vers un état d'alerte maximum, soit vers l'état de blocage immédiat DANGER). Aucun chevauchement.

Conséquences sur l'exécution :

Unicité : le processus de réduction remonte des feuilles vers la racine de manière totalement univoque. L'absence d'ambiguïté élimine tout besoin de retour en arrière (backtracking).

Linéarité temporelle $O(n)$: le calcul de l'état d'un nœud s'effectue en temps constant $O(1)$ dès lors que les états de ses enfants directs sont résolus. Le coût global est donc strictement proportionnel au nombre de nœuds n de l'arbre.

Q3.2 La frontière ne suffit pas

Deux arbres de même frontière et de classes différentes (frontière : « na pend ») :

Arbre n°1 — PREFIXED	Arbre n°2 — SUFFIXED
word	word
/ \	/ \
prefixes rest	rest suffixes
prefix na root pend	root na suffix pend

Deux arbres de même frontière et de classes différentes :

Considérons la frontière textuelle plate « na pend » lue de gauche à droite.

Deux arbres syntaxiques distincts partagent cette même frontière :

- L'arbre **préfixé** : un nœud global qui associe un groupe de préfixes « na » à un radical nu « pend ».
- L'arbre **suffixé** : un nœud global qui associe un radical nu « na » à un groupe de suffixes « pend ».

Échec de l'AFD de mots : L'automate fini linéaire du Jour 1 ne perçoit le flux d'entrée que comme une chaîne de caractères unidimensionnelle. Face à la même séquence linéaire de symboles, un AFD de mots se trouve dans

l'incapacité d'isoler les deux structures parenthésées sous-jacentes. Il est structurellement aveugle aux regroupements hiérarchiques profonds et ne peut pas attribuer des étiquettes sémantiques distinctes à ces deux configurations.

Q3.3 Théorème du yield

Démonstration pour les mots morphologiques valides : L'automate morphologique n'accepte un arbre au niveau de sa racine que si sa structure valide une imbrication bien ordonnée. Un mot complet se décompose obligatoirement en une séquence (éventuellement vide) de préfixes, suivie d'un radical central unique, suivie d'une séquence (éventuellement vide) de suffixes.

- La projection ordonnée des feuilles du sous-arbre gauche engendre exclusivement un langage de la forme p^* (itérations du jeton préfixe).
- Le nœud central fournit l'unique élément du radical r .
- La projection ordonnée des feuilles du sous-arbre droit engendre exclusivement un langage de la forme s^* (itérations du jeton suffixe).

La frontière de tout arbre accepté est donc de la forme :

$$p^* \cdot r \cdot s^*$$

Lien avec le Jour 1 : L'extraction de la frontière (yield) de ces arbres réguliers produit exactement le langage formel p^*rs^* . En projetant l'alphabet abstrait du Jour 1 (où le préfixe équivaut à la lettre a , le radical à o et le suffixe à r), on retrouve le langage rationnel régulier a^*or^* . Cela illustre physiquement le théorème selon lequel l'ensemble des frontières d'un langage d'arbres régulier forme un langage de mots régulier.

Q3.4 Réduplication

Réponse : Non, la réduplication n'engendre pas un langage d'arbres régulier.

Justification : La réduplication morphologique impose une contrainte de copie stricte de la forme $\{ww \mid w \in \Sigma^*\}$. Or, un automate d'arbres régulier (BUTA) opère de manière purement locale et ascendante avec un nombre fini d'états. Il est incapable de propager ou de mémoriser une chaîne de caractères arbitrairement longue d'une branche de l'arbre à une autre pour forcer une identité parfaite entre deux sous-arbres éloignés. Comme l'a démontré Culy sur le Bambara (Culy, 1985), la réduplication rompt la régularité et sort même du cadre des langages hors-contexte, nécessitant des formalismes de grammaires d'arbres beaucoup plus puissants (comme les extensions indexées).

Q3.5 —Rapport sur la compression par Hash-Consing

Formule mathématique de la compression :

$$\text{Taux} = 1 - (\text{Nombre de nœuds uniques stockés} / \text{Nombre total de nœuds générés})$$

Analyse typologique des familles de langues :

Type de langue	Exemple	Structure morphologique	Taux attendu
Agglutinante	Turc, Finnois	Affixes en cascade sur racine stable	Élevé
Isolante	Anglais	Mots courts, quasi pas d'affixes	Faible
Flexionnelle	Latin, Russe	Affixes fusionnels, racines variables	Moyen

Langues agglutinantes (Turc, Finnois) : Ce sont elles qui présentent le taux de compression le plus élevé. La morphologie de ces langues repose sur l'affixation en cascade de morphèmes grammaticaux standardisés (marques de pluriel, de possession, de cas) autour d'une racine stable. Par conséquent,

de très nombreux mots partagent exactement les mêmes structures de sous-arbres de suffixes. Le hash-consing élimine ces redondances en fusionnant ces sous-arbres en un unique Graphe Orienté Acyclique (DAG), maximisant ainsi le gain mémoire.

Langues isolantes (Anglais) : Ce sont elles qui affichent le taux de compression le plus faible. Les mots y sont structurellement courts, invariants, et les relations grammaticales s'expriment par des mots fonctionnels indépendants plutôt que par des modifications morphologiques lourdes. Les arbres syntaxiques associés se réduisent à des structures quasi-plates ou atomiques, offrant très peu de motifs répétitifs ou de sous-arbres complexes à factoriser.

```
>> from apps.morpho.automaton import build_word
>> from apps.hashcons.store import CompactStore
>> from apps.morpho.corpora import corpus_A, corpus_B
>> from apps.morpho.discover import discover, segment_to_tree
>>
>> s = CompactStore()
>> vocab_A = set(corpus_A())
>> vocab_B = set(corpus_B())
>> PRE = discover(vocab_A, prefix_side=True)
>> SUF = discover(vocab_B, prefix_side=False)
>>
>> for w in corpus_A() + corpus_B():
>>     tree = segment_to_tree(w, PRE, SUF)
>>     s.intern(tree)
>>
>> print('Noeuds totaux      : ', s.total_nodes())
>> print('Noeuds uniques    : ', s.unique_nodes())
>> print('Taux compression : ', round(s.compression() * 100, 2), '%')
```

Résultats obtenus sur le corpus (corpus_A + corpus_B) :

```
Noeuds totaux      : 18300
Noeuds uniques     : 4083
Taux compression   : 77.69 %
```

Interprétation : Sur 18 300 nœuds générés, seulement 4 083 sont stockés réellement. Le hash-consing élimine donc 77,69 % des nœuds redondants. Ce

taux élevé s'explique par le fait que les corpus utilisés (corpus_A à préfixes, corpus_B à suffixes) partagent massivement les mêmes sous-arbres de préfixes et suffixes autour de racines communes — comportement typique des langues agglutinantes, confirmant l'analyse théorique.

E3.1 Moteur BUTA

run parcourt l'arbre depuis les feuilles vers la racine (post-ordre). Pour chaque nœud, on calcule d'abord l'état de chacun de ses enfants. Si un enfant renvoie REJECT on s'arrête immédiatement et on rejette tout. Sinon on cherche dans la table de règles si la combinaison (symbole du nœud + états des enfants) existe. Si elle existe on renvoie l'état résultat, sinon REJECT.

accepts appelle simplement **run** et vérifie si l'état obtenu à la racine fait partie des états finaux.

Code source: tree.py : run + accepts

```
def run(self, t: "Term"):  
    child_states = []  
    for child in t.children:  
        state = self.run(child)  
        if isinstance(state, _Reject):  
            return REJECT  
        child_states.append(state)  
  
    # Chercher la règle correspondante  
    key = (t.symbol, tuple(child_states))  
    if key not in self.delta:  
        return REJECT  
    return self.delta[key]  
  
def accepts(self, t: "Term") -> bool:  
    return self.run(t) in self.final
```

E3.2 Automate morphologique

morpho_automaton définit toutes les règles qui décrivent une structure de mot valide. On part des feuilles : une feuille **prefix** donne l'état PRE, **root** donne ROOT, **suffix** donne SUF, **nil** donne NIL. Ensuite on monte : une chaîne de préfixes donne PREFS, une chaîne de suffixes donne SUFS, un radical avec ses suffixes donne REST, et enfin un mot complet (préfixes + REST) donne WORD étant le seul état final accepté.

Code source:

```
def morpho_automaton() -> TreeAutomaton:
    A = TreeAutomaton(final_states={"WORD"})

    # Feuilles
    A.add_rule("nil", (), "NIL")
    A.add_rule("prefix", (), "PRE")
    A.add_rule("root", (), "ROOT")
    A.add_rule("suffix", (), "SUF")

    # Chaînes de préfixes et suffixes
    A.add_rule("prefixes", ("PRE", "NIL"), "PREFS")
    A.add_rule("prefixes", ("PRE", "PREFS"), "PREFS")
    A.add_rule("suffixes", ("SUF", "NIL"), "SUFS")
    A.add_rule("suffixes", ("SUF", "SUFS"), "SUFS")

    # rest et word
    A.add_rule("rest", ("ROOT", "NIL"), "REST")
    A.add_rule("rest", ("ROOT", "SUFS"), "REST")
    A.add_rule("word", ("NIL", "REST"), "WORD")
    A.add_rule("word", ("PREFS", "REST"), "WORD")

    return A
```

automaton.py : morpho_automaton

E3.3 AttackDecomposer

_seq calcule le niveau de danger d'une séquence de deux éléments. Si l'un des deux est déjà DANGER → DANGER. Si on a OVR et ROLE ensemble → DANGER (combinaison interdite). Sinon on garde le niveau de sévérité le plus

élevé des deux. **shield_automaton** construit le pare-feu complet. Les feuilles reçoivent leur niveau de base : **txt** et **enc** sont SAFE, **ovr** est OVR, **role** est ROLE. **frame** et **sys** font monter tout ce qui est OVR, ROLE ou DANGER directement à DANGER. **seq** applique **_seq** pour toutes les combinaisons possibles d'états.

Code source

```
def _seq(x, y):
    if x == DANGER or y == DANGER:
        return DANGER
    if {x, y} == {OVR, ROLE}:
        return DANGER
    if x == DANGER or y == DANGER:
        return DANGER
    # sinon on prend la sévérité maximale
    sx = _SEV.get(x, 0)
    sy = _SEV.get(y, 0)
    return _BY_SEV[max(sx, sy)]

def shield_automaton() -> TreeAutomaton:
    A = TreeAutomaton(final_states={DANGER})

    # Feuilles
    A.add_rule("txt", (), SAFE)
    A.add_rule("enc", (), SAFE)
    A.add_rule("ovr", (), OVR)
    A.add_rule("role", (), ROLE)

    # seq : produit cartésien de tous les états
    for x in _ALL:
        for y in _ALL:
            A.add_rule("seq", (x, y), _seq(x, y))

    # frame et sys : propagent et escaladent vers DANGER
    for x in _ALL:
        A.add_rule("frame", (x,), DANGER if x in (OVR, ROLE, DANGER) else SAFE)
        A.add_rule("sys", (x,), DANGER if x in (OVR, ROLE, DANGER) else SAFE)

    return A
```

decomposer.py : **_seq** + **shield_automaton**

E3.4 Produit

product construit un automate qui accepte uniquement les arbres acceptés par les deux automates en même temps. Chaque état du nouvel automate est une paire d'états (un de chaque automate). On ne garde que les règles où les deux automates traitent le même symbole avec le même nombre d'enfants. Un état est final uniquement si les deux composantes sont finales dans leurs automates respectifs.

```
def product(a1: "TreeAutomaton", a2: "TreeAutomaton") -> "TreeAutomaton":
    final_pairs = {(f1, f2) for f1 in a1.final for f2 in a2.final}
    p = TreeAutomaton(final_states=final_pairs)

    for (sym1, kids1), r1 in a1.delta.items():
        for (sym2, kids2), r2 in a2.delta.items():
            if sym1 == sym2 and len(kids1) == len(kids2):
                paired_kids = tuple(zip(kids1, kids2))
                p.add_rule(sym1, paired_kids, (r1, r2))

    return p
```

tree.py : product

E3.5 Hash-consing

intern stocke chaque nœud de l'arbre une seule fois. On commence par interner récursivement tous les enfants pour obtenir leurs identifiants. Puis on construit une clé unique à partir du symbole, du label et des identifiants des enfants. Si cette clé existe déjà dans la table, on retourne l'identifiant existant. Sinon on crée un nouvel identifiant.

get fait l'inverse : à partir d'un identifiant on reconstruit l'arbre complet en récupérant récursivement les enfants.

compression mesure le gain obtenu grâce au partage : $1 - (\text{noeuds uniques} / \text{noeuds total})$. Plus il y a de sous-arbres répétés, plus le taux est élevé.

Code source :

```
def intern(self, t: Term) -> NodeId:
    kids_ids = tuple(self.intern(child) for child in t.children)
    self._total += 1
    key = (t.symbol, t.label, kids_ids)
    if key in self._table:
        return self._table[key]
    nid = len(self._nodes)
    self._nodes.append((t.symbol, t.label, kids_ids))
    self._table[key] = nid
    return nid

def get(self, nid: NodeId) -> Term:
    symbol, label, kids_ids = self._nodes[nid]
    children = tuple(self.get(kid) for kid in kids_ids)
    return Term(symbol, children, label)

def compression(self) -> float:
    if self._total == 0:
        return 0.0
    return 1 - self.unique_nodes() / self.total_nodes()
```

store.py : intern + get + compression

E3.6 Double encodage (bonus)

enc_automaton compte le nombre de feuilles **enc** dans un arbre, plafonné à 2. Les autres feuilles comptent 0, **enc** compte 1. **seq** additionne les deux compteurs (sans dépasser 2), **frame** et **sys** propagent simplement le compteur. L'état final est 2 donc on n'accepte que les arbres avec au moins 2 **enc**.

dangerous_and_double_encoded fait le produit des deux automates : on bloque uniquement ce qui est à la fois dangereux (shield) et doublement encodé (enc_automaton).

```

def enc_automaton() -> TreeAutomaton:
    A = TreeAutomaton(final_states={2})

    A.add_rule("txt", (), 0)
    A.add_rule("ovr", (), 0)
    A.add_rule("role", (), 0)
    A.add_rule("enc", (), 1)

    for x in (0, 1, 2):
        for y in (0, 1, 2):
            A.add_rule("seq", (x, y), min(2, x + y))
            A.add_rule("frame", (x,), x)
            A.add_rule("sys", (x,), x)

    return A

def dangerous_and_double_encoded() -> TreeAutomaton:
    from formlang.tree import product
    return product(shield_automaton(), enc_automaton())

```

decomposer.py : enc_automaton + dangerous_and_double_encoded

Résultats des tests

```

tests/test_tree.py::test_morpho_accept_et_classes PASSED [ 12%]
tests/test_tree.py::test_morpho_rejet PASSED [ 25%]
tests/test_tree.py::test_shield_blocage PASSED [ 37%]
tests/test_tree.py::test_produit_intersection PASSED [ 50%]
tests/test_tree.py::test_shield_double_encodage_P45 PASSED [ 62%]
tests/test_hashcons.py::test_round_trip_exact PASSED [ 75%]
tests/test_hashcons.py::test_sous_arbres_identiques_partages PASSED [ 87%]
tests/test_hashcons.py::test_compression_positive PASSED [100%]
===== 8 passed in 0.10s =====

```

```

PS C:\Users\RUBENS\Documents\Master\S
== démo Shield (AttackDecomposer) ==
OK      seq(txt,txt)
OK      role (isolé)
BLOQUÉ  sys(role)
BLOQUÉ  seq(frame(ovr),txt)
BLOQUÉ  sys(seq(txt,frame(role)))

```

Bonus Jour 3 infixation, minimisation BUTA

Infixation : on ajoute une nouvelle façon de couper une racine en trois (racine-gauche + infixe + racine-droite) et une règle d'automate qui reconnaît ce nouveau motif, en plus des préfixes/suffixes déjà connus.

- *Minimisation BUTA* : c'est la même idée que Moore mais sur des arbres. On sépare d'abord les états finaux des non-finaux. Puis, pour chaque état, on regarde sa "signature" : dans quelles règles il apparaît, à quelle position, et vers quel groupe mènent ses partenaires et le résultat. Deux états avec la même signature sont fusionnés. On répète jusqu'à stabilité, puis on reconstruit un automate plus petit avec un représentant par groupe.
- *P4.5* : on construit un second automate qui compte juste le nombre de feuilles enc (plafonné à 2), puis on fait le produit avec l'automate Shield : un arbre n'est bloqué que s'il est à la fois dangereux ET doublement encodé — exemple concret de la clôture par intersection.

```
def minimize_buta(a: "TreeAutomaton") -> "TreeAutomaton":
    """Minimisation d'un BUTA par raffinement de partition."""

    # Collecter tous les états
    states = set(a.final)
    for (sym, kids), res in a.delta.items():
        states.add(res)
        for k in kids:
            states.add(k)

    # Partition initiale : finaux vs non-finaux
    finals = frozenset(s for s in states if s in a.final)
    non_finals = frozenset(s for s in states if s not in a.final)
    partition = []
    if finals: partition.append(finals)
    if non_finals: partition.append(non_finals)

    def block_id(s):
        for i, b in enumerate(partition):
            if s in b:
                return i
        return -1
```


formlang/tree.py :fonction minimize_buta

```
def test_infixation():
    from apps.morpho.automaton import (
        morpho_automaton_with_infix, build_word_infix, classify
    )
    A = morpho_automaton_with_infix()
    # mot infixé : racine "str" + infixe "um" + racine "ing" -> "strumming"
    t = build_word_infix([], "str", "um", "ing", [])
    assert classify(A, t) == "INFIXED"
    # avec préfixe en plus
    t2 = build_word_infix(["re"], "str", "um", "ing", [])
    assert classify(A, t2) == "INFIXED"
```

```
def test_minimisation_buta():
    from formlang.tree import TreeAutomaton, minimize_buta, Term

    A = TreeAutomaton(final_states={"q1", "q2"})
    A.add_rule("f", (), "q1")
    A.add_rule("g", (), "q2")
    A.add_rule("a", (), "q0")
    A.add_rule("b", (), "q3")

    M = minimize_buta(A)
    all_states = set()
    for (_, _), res in M.delta.items():
        all_states.add(res)
    all_states |= M.final

    assert len(all_states) == 2 # un état final + un état non-final
    assert M.accepts(Term("f")) is True
    assert M.accepts(Term("g")) is True
    assert M.accepts(Term("a")) is False
    assert M.accepts(Term("b")) is False
```

tests/test_tree.py : test test_infixation tests/test_tree.py : test

test_minimisation_buta

```
tests/test_tree.py::test_infixation PASSED [ 50%]
tests/test_tree.py::test_minimisation_buta PASSED [100%]

===== 2 passed in 0.06s =====
```

Jour 4 Calculabilité : machine de Turing & machine universelle

Q4.1 Encodage $\langle M \rangle$

Schéma d'encodage canonique

Pour linéariser une Machine de Turing $M = (Q, \Sigma, \Gamma, \delta, q_0, \sqcup, F)$ sous forme de chaîne $\langle M \rangle$, on applique un codage unaire strict :

Objet encodé	Encodage unaire
État $q_i \in Q$	1^i (i répétitions du symbole 1)
Symbole $X_{\square} \in \Gamma$	1^j
Mouvement L / R / S	1 / 11 / 111
Transition $\delta(q_i, X_{\square}) = (q_{\square}, X_{\square}, D)$	$1^i 0 1^j 0 1^k 0 1^l 0 1^D$ (quintuplet)
Séparateur entre quintuplets	00

Exemple : $\delta(q_1, X_2) = (q_3, X_1, R)$ s'encode : 1 0 11 0 111 0 1 0 11

La chaîne globale $\langle M \rangle$ est la concaténation de tous les quintuplets séparés par 00.

Preuve d'injectivité et de décodabilité :

Injectivité : Le codage associe à chaque table de transition distincte une chaîne unique. Le délimiteur unique 0 empêche tout chevauchement d'unaires. Deux machines différentes produisent nécessairement deux chaînes binaires distinctes.

Décodabilité : L'analyseur de la machine universelle lit le ruban de gauche à droite. Il isole chaque bloc délimité par 00. À l'intérieur d'un bloc, il compte le nombre de 1 séparés par de simples 0. Cela permet de reconstruire l'état source, le symbole lu, l'état cible, le symbole écrit et le déplacement sans aucune ambiguïté.

Q4.2 La machine universelle U

Invariant du ruban

Le ruban de la machine universelle U est structuré en trois zones distinctes, séparées par des marqueurs ## :

Ruban(U) : $\langle M \rangle \## q_{sim} \## w_{sim}$

$\langle M \rangle$: Description immuable de la table de transition de M.

q_{sim} : État actuel de la machine simulée (mis à jour à chaque cycle).

w_{sim} : Contenu actuel du ruban simulé, avec un pointeur sur la position de la tête virtuelle.

Cycle d'exécution complet (5 étapes) :

Étape	Action de U
1. Lecture	U examine le symbole pointé sur le ruban virtuel w_{sim}
2. Recherche	U compare le couple $(q_{sim}, \text{symbole_lu})$ avec les quintuplets dans $\langle M \rangle$
3. Écriture	Une fois la transition trouvée, U modifie le symbole sur w_{sim}
4. Déplacement	U déplace son pointeur de tête virtuelle sur w_{sim} (L, R ou S)
5. Mise à jour	U réécrit le nouvel état dans la zone q_{sim} et entame le cycle suivant

Argument de correction par récurrence :

Au pas $t = 0$, le ruban contient $\langle M \rangle \## q_0 \## w$. L'état virtuel est bien l'état initial, le ruban virtuel correspond exactement au mot d'entrée.

Supposons que l'invariant soit maintenu au pas t . Au pas $t+1$, le cycle de U applique précisément la transition unique $\delta(q_i, X_i)$ définie par la spécification de M. L'état, le symbole et la position de la tête après

écriture sont identiques à ceux d'une exécution native de M . L'invariant est conservé au pas $t+1$.

Par récurrence, U simule fidèlement le comportement exact de M pour tout nombre de pas fini.

Q4.3 Surcoût de la simulation

Le passage d'une exécution native à une exécution universelle engendre des ralentissements structurels précis :

Configuration	Complexité temporelle	Source
U multi-ruban simulant M	$O(t \cdot M)$	Sipser, Thm 7.8
Transformation multi-ruban $\rightarrow$ mono-ruban	$O(t^2 \cdot M ^2)$	Hopcroft & Ullman §7
Simulation optimisée 2-rubans (Hennie–Stearns)	$O(t \log t)$	J. ACM 13(4), 1966

Complexité multi-ruban : Pour une machine M s'arrêtant en t étapes, le coût sur une machine universelle multi-ruban est de $O(t \cdot |M|)$. Le facteur multiplicatif $|M|$ provient du temps requis pour parcourir la table des transitions à chaque pas de calcul.

Transformation multi-ruban vers mono-ruban : La simulation des rubans de contrôle sur un unique ruban physique induit un surcoût quadratique classique, portant la complexité à $O(t^2 \cdot |M|^2)$.

Simulation optimisée (Hennie–Stearns, 1966) : En appliquant l'architecture de calcul établie par F. C. Hennie et R. E. Stearns (J. ACM 13(4)

:533–546, 1966), la simulation d'une machine à k rubans par une machine à 2 rubans s'effectue avec un ralentissement seulement logarithmique. Le coût temporel optimal s'élève à $O(t \log t)$.

Q4.4 Indécidabilité

Réduction depuis le problème de l'arrêt (HALT)

Le problème de l'arrêt **HALT** = $\{ \langle M \rangle, w \mid M \text{ s'arrête sur } w \}$ est historiquement indécidable (Turing, 1936).

Construisons une réduction vers le langage de la machine universelle :

$$L_UTM = \{ \langle M \rangle \#\# w \mid U \text{ s'arrête sur } \langle M \rangle \#\# w \}$$

Par définition, l'évaluation de $U(\langle M \rangle \#\# w)$ imite fidèlement l'exécution de $M(w)$. Par conséquent :

$$U \text{ s'arrête sur } \langle M \rangle \#\# w \Leftrightarrow M \text{ s'arrête sur } w$$

Si un algorithme décideur existait pour L_UTM , il suffirait de lui soumettre la chaîne $\langle M \rangle \#\# w$ pour trancher immédiatement l'arrêt de M sur w . Cela résoudrait HALT, ce qui est **contradictoire**. L'arrêt de U est donc strictement indécidable.

Commentaire : « Universel $\neq$ Omniscient »

Cette indécidabilité illustre la nuance fondamentale du théorème de Turing :

Terme	Signification
Universel	Possède la puissance de calcul maximale : capacité de reproduire n'importe quel algorithme mécanisable
Omniscient	Pourrait analyser et prédire statiquement le comportement à long terme des programmes qu'il exécute

La machine universelle peut tout calculer, mais elle est incapable de prédire à l'avance si sa propre simulation finira par s'arrêter. L'universalité logicielle s'accompagne obligatoirement d'une perte totale de la décidabilité globale

E4.1 MT générique

run exécute la machine de Turing pas à pas. Le ruban est représenté comme un dictionnaire où chaque position pointe vers un symbole. La tête de lecture démarre à la position 0 dans l'état initial. À chaque pas : on lit le symbole sous la tête, on vérifie si on est dans un état final (arrêt accepté), on cherche la transition correspondante (arrêt rejeté si elle n'existe pas), puis on écrit le nouveau symbole, on change d'état et on déplace la tête à gauche ou à droite. Si la machine tourne trop longtemps sans s'arrêter, on lève une erreur après `max_steps` pas.

```
def run(self, word: str, max_steps: int = 1_000_000, trace: bool = False) -> "TMResult":
    tape = {i: c for i, c in enumerate(word)}
    head = 0
    state = self.start
    steps = 0
    history = []

    while steps < max_steps:
        symbol = tape.get(head, self.blank)

        if trace:
            history.append((self._window(tape), state, head))

        # Arrêt sur état final
        if state in self.accept:
            return TMResult(True, self._read(tape), steps, history)

        # Arrêt si pas de transition
        key = (state, symbol)
        if key not in self.transitions:
            return TMResult(False, self._read(tape), steps, history)

        # Appliquer la transition
        new_state, new_symbol, direction = self.transitions[key]
        tape[head] = new_symbol
        state = new_state
        if direction == "R":
            head += 1
        elif direction == "L":
            head -= 1
        steps += 1

    raise RuntimeError(f"Machine non arrêtée après {max_steps} pas")
```

turing.py : run

E4.2 Machine universelle

encode transforme une machine de Turing en chaîne de texte JSON. Chaque transition (état, symbole lu) $\rightarrow$ (état suivant, symbole écrit, direction) est sérialisée sous forme de clé-valeur. La chaîne obtenue est injective : deux machines différentes donnent deux chaînes différentes.

decode fait l'inverse : elle relit la chaîne JSON et reconstruit exactement la même machine avec ses transitions, son état initial et ses états finaux.

UniversalTM.run reçoit une machine encodée et un mot. Elle décode la machine puis la fait tourner normalement via **run**. C'est l'interpréteur universel : il n'a pas besoin de connaître la machine à l'avance.

```
def encode(machine: "TuringMachine") -> str:
    data = {
        "transitions": {
            json.dumps([q, a]): [q2, b, d]
            for (q, a), (q2, b, d) in machine.transitions.items()
        },
        "start": machine.start,
        "accept": sorted(machine.accept),
        "blank": machine.blank,
    }
    return json.dumps(data, sort_keys=True)

def decode(desc: str) -> "TuringMachine":
    data = json.loads(desc)
    transitions = {
        tuple(json.loads(k)): tuple(v)
        for k, v in data["transitions"].items()
    }
    return TuringMachine(
        transitions=transitions,
        start=data["start"],
        accept=set(data["accept"]),
        blank=data["blank"],
    )

class UniversalTM:
    def run(self, encoded_machine: str, word: str, **kw) -> "TMResult":
        machine = decode(encoded_machine)
        return machine.run(word, **kw)
```

utm.py : encode + decode + UniversalTM.run

E4.3 Tables ADD et SUB

ADD fonctionne en 3 phases : avancer jusqu'au + et le remplacer par 1 (fusionnant les deux blocs), avancer jusqu'à la fin du ruban, puis effacer le dernier 1 (pour corriger le + transformé en 1). Le résultat est le nombre de 1 restants.

SUB est plus complexe. Elle marque un 1 de m avec X, traverse jusqu'à n, marque un 1 de n avec X, et revient au début. On répète jusqu'à épuisement de n. Quand n est épuisé, elle remonte traverser les X de n et le - pour retrouver le dernier X de m et le remettre en 1. Puis elle efface les X consommés de m, le -, et les X de n. Les 1 non consommés de m restent : c'est le résultat.

addition encode les deux nombres en unaire ($1^n + 1^m$) et passe le ruban à **ADD**. Elle compte les 1 du résultat.

soustraction gère les cas limites ($m = 0 \rightarrow$ retourner n, $m \geq n \rightarrow$ retourner 0) puis passe le ruban à **SUB**.

multiplication répète l'addition m fois en chaînant les résultats.

division soustrait m de n en boucle jusqu'à ce que le reste soit inférieur à m, en comptant le nombre de soustractions (quotient).

chaîner applique une liste d'opérations en séquence, en passant le résultat de chacune à la suivante.


```

ADD = TuringMachine(
    transitions={
        #avancer jusqu'au +, le remplacer par 1
        ("q0", "1"): ("q0", "1", "R"),
        ("q0", "+"): ("q1", "1", "R"),
        #avancer jusqu'à la fin
        ("q1", "1"): ("q1", "1", "R"),
        ("q1", "_"): ("q2", "_", "L"),
        #effacer le dernier 1
        ("q2", "1"): ("qf", "_", "S"),
    },
    start="q0", accept={"qf"},
)

```

```

SUB = TuringMachine(
    transitions={
        # Marquer un 1 de m avec X, aller chercher un 1 de n
        ("q0", "1"): ("q1", "X", "R"),
        ("q0", "X"): ("q0", "X", "R"),
        ("q0", "-"): ("q_clean", "-", "L"), # m épuisé

        # Traverser jusqu'à n
        ("q1", "1"): ("q1", "1", "R"),
        ("q1", "X"): ("q1", "X", "R"),
        ("q1", "-"): ("q2", "-", "R"),

        # Marquer un 1 de n avec X
        ("q2", "X"): ("q2", "X", "R"),
        ("q2", "1"): ("q3", "X", "L"),
        ("q2", "_"): ("q_back", "_", "L"), # n épuisé avant m

        # Revenir au début
        ("q3", "X"): ("q3", "X", "L"),
        ("q3", "1"): ("q3", "1", "L"),
        ("q3", "-"): ("q3", "-", "L"),
        ("q3", "_"): ("q0", "_", "R"),

        # n épuisé : traverser X de n et - pour trouver le dernier X de m
        ("q_back", "X"): ("q_back", "X", "L"),
        ("q_back", "-"): ("q_back2", "-", "L"),

        # Remettre le dernier X de m en 1
        ("q_back2", "1"): ("q_back2", "1", "L"),
    }
)

```

machines.py : tables ADD et SUB

```

class Calculatrice:
    def addition(self, n: int, m: int) -> int:
        tape = "1" * n + "+" + "1" * m
        return ADD.run(tape).tape.count("1")

    def soustraction(self, n: int, m: int) -> int:
        if m == 0:
            return n
        if m >= n:
            return 0
        tape = "1" * n + "-" + "1" * m
        return SUB.run(tape).tape.count("1")

    def multiplication(self, n: int, m: int) -> int:
        result = 0
        for _ in range(m):
            result = self.addition(result, n)
        return result

    def division(self, n: int, m: int):
        if m == 0:
            raise ZeroDivisionError("division par zéro")
        quotient, reste = 0, n
        while reste >= m:
            reste = self.soustraction(reste, m)
            quotient = self.addition(quotient, 1)
        return quotient, reste

    def chainer(self, v0: int, ops: list) -> int:
        result = v0

```

calculator.py : Calculatrice

E4.4 Interpréteur universel

UniversalInterpreter.run encode la machine reçue en paramètre avec **encode**, puis délègue l'exécution à **UniversalTM.run**. Il ne réécrit aucune boucle d'exécution.

addition_via_utm et **soustraction_via_utm** utilisent **UniversalInterpreter** pour faire tourner **ADD** et **SUB** via la machine universelle, au lieu de les appeler directement. C'est la démonstration concrète qu'une machine universelle peut exécuter n'importe quelle autre machine à partir de sa description.

```

class UniversalInterpreter:
    def __init__(self):
        self._U = UniversalTM()

    def run(self, machine, word, **kw):
        encoded = encode(machine)
        return self._U.run(encoded, word, **kw)

def addition_via_utm(n: int, m: int) -> int:
    interp = UniversalInterpreter()
    tape = "1" * n + "+" + "1" * m
    return interp.run(ADD, tape).tape.count("1")

def soustraction_via_utm(n: int, m: int) -> int:
    if m == 0:
        return n
    if m >= n:
        return 0
    interp = UniversalInterpreter()
    tape = "1" * n + "-" + "1" * m
    return interp.run(SUB, tape).tape.count("1")

```

interpreter.py : UniversalInterpreter + addition_via_utm

Résultats des tests

```

tests/test_turing.py::test_round_trip_encodage PASSED [ 12%]
tests/test_turing.py::test_utm_simule_comme_la_machine PASSED [ 25%]
tests/test_turing.py::test_trace_disponible PASSED [ 37%]
tests/test_turing.py::test_interpreteur_universel PASSED [ 50%]
tests/test_calc.py::test_add_sub_machines_directes PASSED [ 62%]
tests/test_calc.py::test_calculatrice_exhaustif PASSED [ 75%]
tests/test_calc.py::test_chainage PASSED [ 87%]
tests/test_calc.py::test_division_par_zero PASSED [100%]

===== 8 passed in 0.11s =====

```

Bonus Jour 4 table monolithique de multiplication

Contrairement à la calculatrice qui fait $m \times n$ en additionnant n fois m (donc en rappelant la machine ADD en boucle depuis Python), ici une seule et unique table de transitions calcule directement le produit. Le ruban contient $1^m * 1^n =$. La machine marque un 1 du premier bloc, puis recopie tout le second bloc à la fin du ruban ; elle répète ça pour chaque 1 du premier bloc, en démarquant à chaque tour. Une fois le premier bloc épuisé, elle nettoie le ruban (efface le second bloc et le $=$, transforme les marqueurs de résultat en vrais 1) et s'arrête. Pour la valider "par le simulateur", cette table est encodée puis exécutée par la machine universelle plutôt que d'être relancée directement, exactement le même principe que `addition_via_utm`.

```
MUL = TuringMachine(
    transitions={
        # q0 : trouver 1 non marqué de m, le marquer X
        ("q0", "1"): ("q1", "X", "R"),
        ("q0", "X"): ("q0", "X", "R"),
        ("q0", "*"): ("qclean", "*", "R"),    # m épuisé : nettoyer

        # q1 : aller après * pour trouver n
        ("q1", "X"): ("q1", "X", "R"),
        ("q1", "1"): ("q1", "1", "R"),
        ("q1", "*"): ("q2", "*", "R"),

        # q2 : marquer un 1 de n avec Y, l'écrire dans résultat
        ("q2", "Y"): ("q2", "Y", "R"),
        ("q2", "1"): ("q3", "Y", "R"),
        ("q2", "="): ("q5", "=", "L"),      # n épuisé ce tour

        # q3 : aller à la fin du résultat (après =) et écrire R
        ("q3", "Y"): ("q3", "Y", "R"),
        ("q3", "1"): ("q3", "1", "R"),
        ("q3", "="): ("q3", "=", "R"),
```

apps/mtu/machines.py : table MUL

```
def multiplication_via_utm(n: int, m: int) -> int:
    |
    interp = UniversalInterpreter()
    tape = "1" * n + "*" + "1" * m + "="
    return interp.run(MUL, tape).tape.count("1")
```

apps/mtu/interpreter.py :fonction multiplication_via_utm

```
def test_mul_monolithique():
    from apps.mtu.machines import MUL
    assert MUL.run("11*111=").tape.count("1") == 6
    assert MUL.run("111*11=").tape.count("1") == 6
    assert MUL.run("1*1111=").tape.count("1") == 4
    assert MUL.run("11111*11=").tape.count("1") == 10

def test_mul_monolithique_via_utm():
    from apps.mtu.interpreter import multiplication_via_utm
    assert multiplication_via_utm(2, 3) == 6
    assert multiplication_via_utm(5, 2) == 10
    assert multiplication_via_utm(0, 4) == 0
    assert multiplication_via_utm(4, 4) == 16
```

tests/test_calc.py : test test_mul_monolithique_via_utm

```
tests/test_calc.py::test_mul_monolithique PASSED [100%]
===== 1 passed in 0.07s =====
```

Jour 5 Intégration & Myhill–Nerode

Q5.1 Théorème de Myhill–Nerode

Énoncé du théorème

Un langage $L \subseteq \Sigma^*$ est régulier **si et seulement si** sa relation d'équivalence de Nerode $\approx_L$ possède un nombre fini de classes d'équivalence (indice fini). Deux mots u et v sont dits équivalents ($u \approx_L v$) si aucun suffixe w ne permet de les distinguer :

$$\forall w \in \Sigma^*, uw \in L \Leftrightarrow vw \in L$$

Lien chiffré avec le Jour 1

- **Nombre de classes de L_1** : exactement **3 classes**.
- **Correspondance** : l'indice de la relation $\approx_L$ égale strictement le nombre d'états de l'AFD minimal.
- **Lien direct** : les 3 classes de Nerode (C_A , C_B , C_C) correspondent de manière bijective aux 3 états (A, B, C) de l'AFD minimal construit au Jour 1 :

Classe de Nerode	Mots représentants	État AFD minimal
C_A	ε, a	A (initial)
C_B	o, ao	B (« armé »)
C_C	or, aor	C (final)

Q5.2 Indistinguabilité et fusion d'états

Le critère des suffixes témoins

Jeu de suffixes témoins choisi: $S = \{\varepsilon, r, or\}$

- **Principe** : deux mots ont la même *signature* s'ils se comportent de manière identique face à un ensemble fini de suffixes critiques S .
- **Rôle** : il sert de filtre minimal pour séparer les étapes de reconnaissance du motif obligatoire or .

La signature d'un mot w est le vecteur booléen :

$$\text{sig}(w) = (ws \in L_1)_{s \in S} = (w\varepsilon \in L_1, wr \in L_1, wor \in L_1)$$

Mot w	$w\varepsilon \in L_1$	$wr \in L_1$	$wor \in L_1$	Signature	Classe
ε	0	0	1	(0, 0, 1)	C_A
a	0	0	1	(0, 0, 1)	C_A
o	0	1	1	(0, 1, 1)	C_B
ao	0	1	1	(0, 1, 1)	C_B
or	1	1	1	(1, 1, 1)	C_C
aor	1	1	1	(1, 1, 1)	C_C

Les 3 signatures distinctes correspondent exactement aux 3 classes donc aux 3 états.

Lien avec la fusion d'états : minimisation de Moore

- **Diagnostic** : deux états d'un automate sont fusionnés lors de la minimisation s'ils sont jugés indistinguables, c'est-à-dire s'ils acceptent exactement les mêmes suffixes.
- **Mécanisme algébrique** : l'algorithme de Moore raffine itérativement les blocs d'états en observant leurs transitions futures, jusqu'à stabilité.

- **Équivalence** : tester si deux états mènent aux mêmes acceptations pour tout suffixe revient exactement à vérifier qu'ils appartiennent à la même classe de Myhill-Nerode.
- **Résultat** : la fusion de tous les états équivalents produit l'unique automate de taille minimale associé au langage , le même AFD minimal à 3 états obtenu au Jour 1, bouclant ainsi le fil rouge du projet.

E5.1 Pipeline

analyze_word fait passer un mot brut à travers trois étapes en chaîne. D'abord **leet_normalize** traduit les chiffres leet en lettres normales via le transducteur du Jour 1. Ensuite **contains_or** vérifie si le mot normalisé contient le facteur or via l'AFD du Jour 1. Enfin **well_parenthesized** vérifie si les délimiteurs sont bien imbriqués via le PDA du Jour 2. Les trois résultats sont retournés dans un dictionnaire.

analyze_morpho analyse la structure morphologique d'un mot. **discover** parcourt le vocabulaire fourni pour identifier les préfixes et suffixes récurrents. **segment_to_tree** découpe le mot en préfixes, racine et suffixes et construit l'arbre correspondant. **morpho_automaton** instancie l'automate d'arbres du Jour 3. Enfin **classify** fait remonter l'arbre dans l'automate et retourne la classe du mot : BARE, PREFIXED, SUFFIXED ou CIRCUMFIXED.


```

def analyze_word(raw: str) -> dict:
    normalise = leet_normalize(raw)
    or_detecte = contains_or(normalise)
    delimiters = well_parenthesized(normalise)
    return {
        "normalisé(FST)": normalise,
        "facteur_or(AFD)": or_detecte,
        "délimiteurs_ok(PDA)": delimiters,
    }

def analyze_morpho(word: str, vocab: set) -> dict:
    PRE = discover(vocab, prefix_side=True)
    SUF = discover(vocab, prefix_side=False)
    tree = segment_to_tree(word, PRE, SUF)
    A = morpho_automaton()
    classe = classify(A, tree)
    return {
        "prefixes": PRE,
        "suffixes": SUF,
        "classe(BUTA)": classe,
    }

```

pipeline.py : analyze_word + analyze_morpho

E5.3 Classes de Nerode

nerode_classes regroupe les mots qui se comportent de façon identique vis-à-vis du langage. Pour chaque mot w , on calcule sa **signature** : un vecteur booléen qui indique, pour chaque suffixe témoin s de **suffixes**, si la concaténation $w + s$ est acceptée par **accepts**. Deux mots ayant la même signature sont dans la même classe — ils ont le même « avenir » face au langage. On regroupe tous les mots par signature et on retourne la liste des groupes.

Pour L_1 avec les suffixes $\{ "", "r", "or", "a" \}$, on retrouve exactement **3 classes**, ce qui confirme le résultat du Jour 1 : l'AFD minimal a bien 3 états.

```
def nerode_classes(accepts, words, suffixes):
    def signature(w):
        return tuple(accepts(w + s) for s in suffixes)

    groups = {}
    for w in words:
        sig = signature(w)
        groups.setdefault(sig, set()).add(w)

    return list(groups.values())
```

myhill_nerode.py : nerode_classes

Résultats des test

```
tests/test_pipeline.py::test_pipeline_word PASSED [ 20%]
tests/test_pipeline.py::test_pipeline_morpho PASSED [ 40%]
tests/test_pipeline.py::test_demo_shield_verdicts PASSED [ 60%]
tests/test_cfg_nerode.py::test_cfg_engendre_des_mots_equilibres PASSED [ 80%]
tests/test_cfg_nerode.py::test_nerode_trois_classes PASSED [100%]

===== 5 passed in 0.34s =====
```

```
PS C:\Users\RUBENS\Documents\Master\SEMESTRE_TWO\THEORIE DES LANGAGES\formlang_projet> python pipeline.py --word 4or
normalisé(FST) : aor
facteur_or(AFD) : True
délimiteurs_ok(PDA) : True
PS C:\Users\RUBENS\Documents\Master\SEMESTRE_TWO\THEORIE DES LANGAGES\formlang_projet> python pipeline.py --morpho faf
aklar
{'prefixes': {'ki', 'ma', 'vi', 'li', 'mu', 'ba', 'wa', 'tu'}, 'suffixes': set(), 'classe(BUTA)': 'BARE'}
```

```
PS C:\Users\RUBENS\Documents\Master\SEMESTRE_TWO\THEORIE DES LANGAGES\formlang_projet> python -m pytest -q
..... [100%]
28 passed in 0.33s
```

Bonus Jour 5 : mesure du hash-consing sur 10 000 mots

On construit deux vocabulaires synthétiques de 10 000 mots chacun : un "agglutinant" où chaque racine se décline avec les mêmes 8 préfixes et 8 suffixes qui reviennent sans cesse, et un "isolant" où chaque mot est une combinaison de racines libres qui ne partagent aucun affixe. On transforme chaque mot en arbre (comme au Jour 3), puis on les range tous dans le

même magasin de hash-consing. Résultat : le vocabulaire agglutinant se compresse à 76,9 % (les sous-arbres de préfixes/suffixes reviennent tout le temps et sont partagés), contre seulement 40 % pour l'isolant (là, seul un nœud vide générique est partagé — un plancher purement structurel qui ne dépend pas de la vraie morphologie).

```
def corpus_agglutinant(n_words: int = 10_000) -> list:

    forms_per_root = 1 + len(PREFIXES_A) + len(SUFFIXES_B) # bare + préfixées + suffixées
    n_roots = -(-n_words // forms_per_root) # ceil
    out = []
    for r in roots(n_roots):
        out.append(r)
        out += [p + r for p in PREFIXES_A]
        out += [r + s for s in SUFFIXES_B]
        if len(out) >= n_words:
            break
    return out[:n_words]

def corpus_isolant(n_words: int = 10_000) -> list:
    |
    rs = roots(2500)
    n = len(rs)
    out, seen = [], set()
    i, j = 0, 0
    while len(out) < n_words:
        w = rs[i % n] + rs[j % n]
        if w not in seen:
            seen.add(w)
            out.append(w)
        i += 1
        j += 7
        if i % n == 0:
            j += 1
    return out[:n_words]
```

apps/morpho/corpora.py

```
def run_benchmark(n_words: int = 10_000) -> list:
    return [
        measure(corpus_agglutinant(n_words), "agglutinant (turc/finnois, synthétique)"),
        measure(corpus_isolant(n_words), "isolant (anglais, synthétique)"),
    ]
```

apps/hashcons/benchmark.py

```

def main(argv=None):
    p = argparse.ArgumentParser(description="Pipeline formlang")
    p.add_argument("--word")
    p.add_argument("--morpho")
    p.add_argument("--hashcons-bench", action="store_true",
                    help="mesure hash-consing sur 10 000 mots (agglutinant vs isolant)")
    p.add_argument("--mul-utm", nargs=2, type=int, metavar=("M", "N"),
                    help="m*n via la table MUL exécutée par la machine universelle")
    args = p.parse_args(argv)
    if args.word:
        for k, v in analyze_word(args.word).items():
            print(f"{k:>22} : {v}")
    if args.morpho:
        from apps.morpho.corpora import corpus_A
        print(analyze_morpho(args.morpho, set(corpus_A())))
    if args.mul_utm:
        from apps.mtu.interpreter import multiplication_via_utm
        m, n = args.mul_utm
        print(f"{m} * {n} (MUL via UTM) = {multiplication_via_utm(m, n)}")
    if args.hashcons_bench:
        from apps.hashcons.benchmark import run_benchmark
        for report in run_benchmark():
            print(report)
            print()
    if not any([args.word, args.morpho, args.hashcons_bench, args.mul_utm]):
        print("== démo Shield (AttackDecomposer) ==")

```

pipeline.py

```

def test_hashcons_benchmark_10000_mots():

    from apps.hashcons.benchmark import run_benchmark

    aggl, isol = run_benchmark(10_000)

    assert aggl.n_words == 10_000 and isol.n_words == 10_000
    assert aggl.n_prefixes > 0 and aggl.n_suffixes > 0
    assert isol.n_prefixes == 0 and isol.n_suffixes == 0
    assert aggl.compression > isol.compression
    assert aggl.elapsed_s < 5.0 and isol.elapsed_s < 5.0

```

tests/test_hashcons.py : test test_hashcons_benchmark

```

tests/test_hashcons.py::test_hashcons_benchmark_10000_mots PASSED [100%]
===== 1 passed in 0.43s =====

```

En somme , on a:

```

PS C:\Users\RUBENS\Documents\Master\SEMESTRE_TWO\THEORIE DES LANGAGES\formlang_projet> python -m pytest -q
.....
35 passed in 0.74s

```